

Decomposing Ontology into an **Ontology Base and its **Interpretation Layer****

Mustafa Jarrar

VUB STARLab

Vrije Universiteit Brussel, Belgium

mjarrar@vub.ac.be

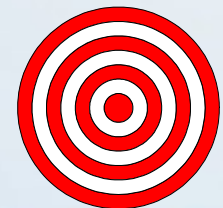
<http://www.starlab.vub.ac.be/staff/mustafa>



VUB STARLab

Problems and Requirements

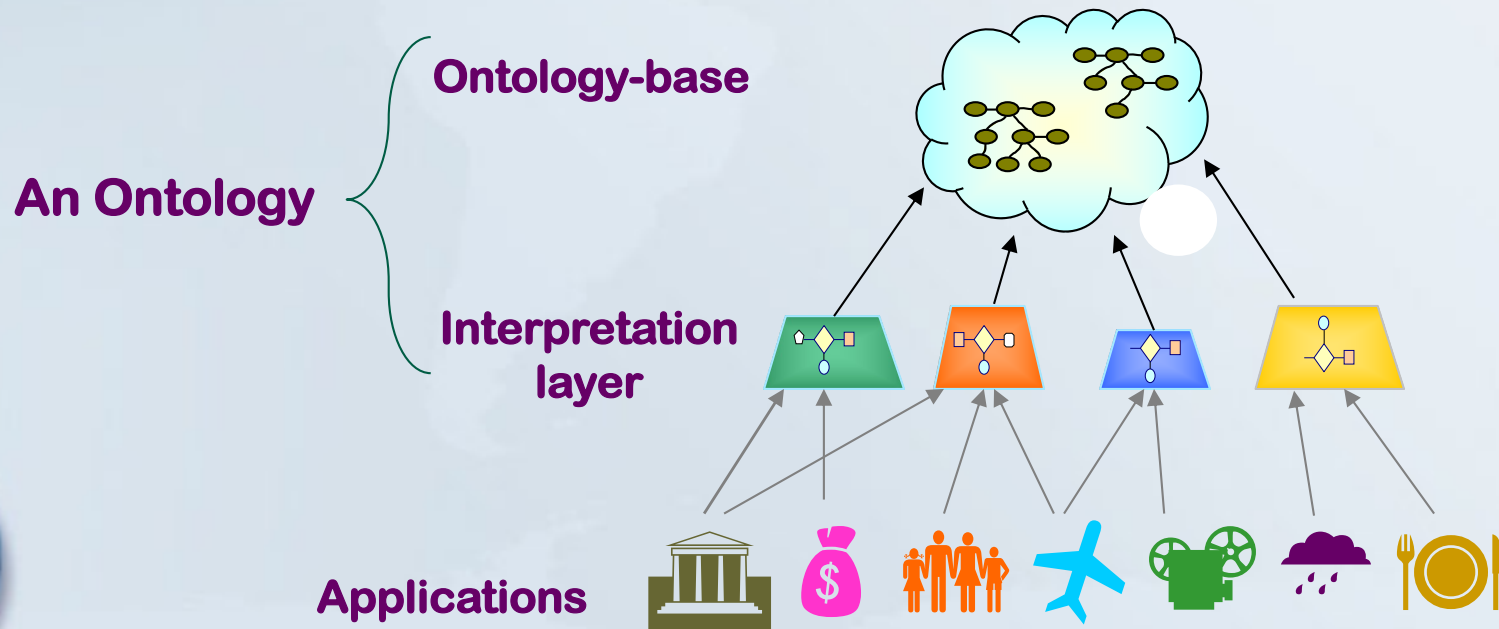
- ✎ Increase independence between ontology and application.
- ✎ Accelerate and simplify ontology construction
- ✎ Build ontology-based business rules.
- ✎ Rules languages and standards should be NL-like, and support legacy standards (e.g. SQL, ER, Java ,...).
- ✎ Manage communication between ontology and applications.
- ✎ Management of large scale ontology bases.
- ✎ Versioning, scalability ,...



The Idea (in short)

Decompose Ontology into: → (called *DOGMA* approach)

- An Ontology Base (set of atomic predicates)
- Interpretation Layer (Rules)



Agenda

- ☐ DOGMA: Definitions
- ☐ Why to decompose
- ☐ Example(1), How to Decompose...
- ☐ Example(2), mining possibilities...
- ☐ Consistency
- ☐ Rule systems
- ☐ Conclusion



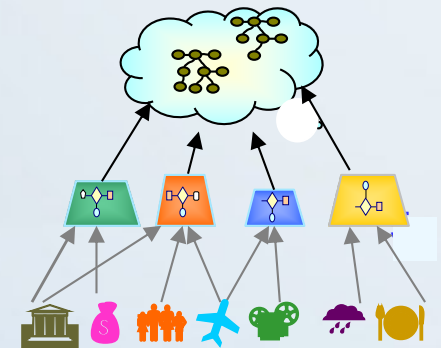
DOGMA: Definitions

Ontology base:

- A set of possible (“*plausible*”) conceptualizations of the real world domain.
- As set of context-specific binary facts (atomic), $\rightarrow$ called *Lexons*. $\rightarrow C(T1, R, T2)$
- Expressed in lexical terms in a given language, in an *intended* sense.
- E.g. : Business (Company, Employs, Employee)
Business (Manager, Is a, Employee)

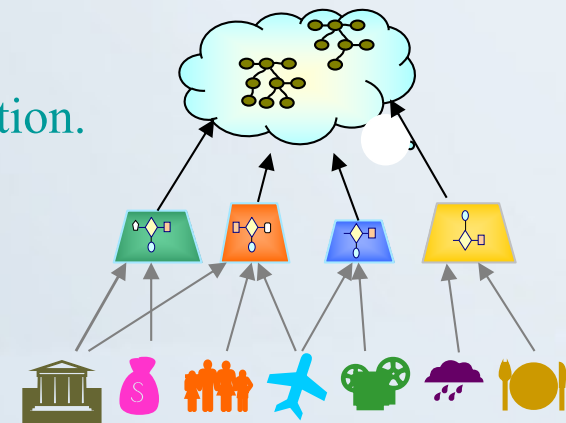
Interpretation layer:

- A separate layer mediating between the ontology base and the application instances committing to the ontology.
- Organized as a set of *Interpretational Views*, where:
 - Each *Interpretational View* is an explicit instance of an *interpretation*
 - A set of rules in a certain syntax (consistent/closed set) that constrain a particular aspect of reality (which is conceptualized in the ontology-base).
 - E.g: Each Manager who Heads a Company must also Works For that Company
- Each application can create (reuse/inherit) its own interpretational view(s);



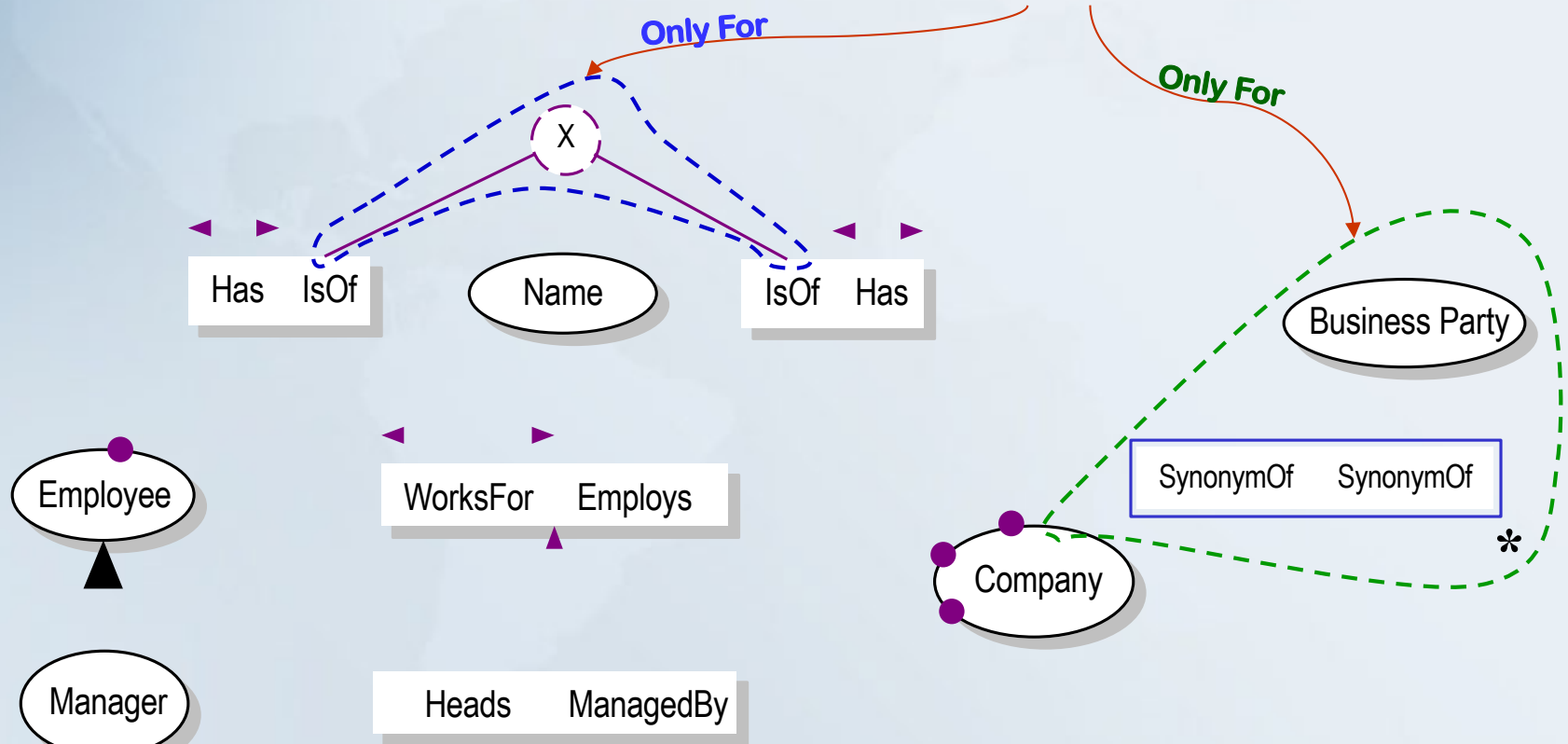
Why: decompose into Ontology-base & Interpretational Layer

- No absolute meaning; everything must be the result of *agreements* among such as designers, domain experts and users.
- All meaning (semantics) is for communication purposes, is represented independent of language but necessarily must be *entirely* rooted and described in (natural) language.
- *Identity* of concepts is a real problem; harder than agreeing about their properties. (E.g identity judgments depend on our conceptualization, [Guarino,98])
- Domain constraints, rules and procedures are essential, but:
 - ❖ Agreement about them is very difficult.
 - ❖ Nearly always specific to a context of application.



Example (1) : Business ontology

Used by two applications A, B



Example (1) : Ontology Base

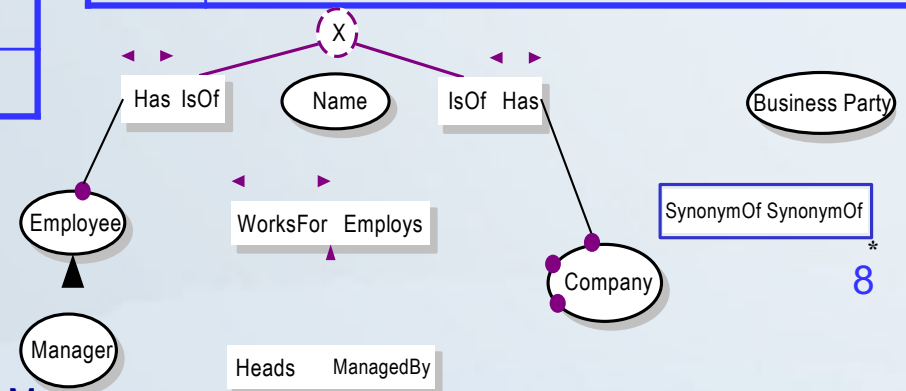
Ontology-base

Lexons

Context	Term1	Role	Term2
Business	Employee	Has	Name
Business	Employee	Works For	Company
Business	Manager	Is a	Employee
Business	Manager	Heads	Company
Business	Company	Has	Name
Business	Company	Managed By	Manager
Business	Company	Employees	Employee
Business	Company	Synonym Of	Business Party
Business	Business Party	Synonym Of	Company
Business	Name	Is Of	Company
Business	Name	Is Of	Employee

Interpretation Layer (rules)

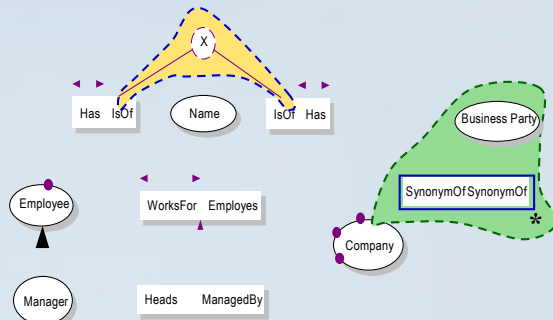
RuleID	Rule
1	Visible Lexons {1,2,3,4,5}
2	Visible Lexons {9,10}
3	Visible Lexons {6,7,8}
4	Each <u>Employee</u> <u>Has</u> at most one <u>Name</u>
5	Each <u>Employee</u> <u>Works For</u> at most one <u>Company</u>
6	Each <u>Employee</u> <u>Has</u> at least one <u>Name</u>
7	Each <u>Company</u> <u>Has</u> at most one <u>Name</u>
8	Each <u>Company</u> <u>Has</u> at least one <u>Name</u>
9	Each <u>Company</u> <u>Employees</u> at least one <u>Employee</u>
10	Each <u>Company</u> <u>Managed By</u> at least one <u>Manager</u>
11	Each <u>Company</u> <u>Managed By</u> at least one <u>Manager</u>
12	Each <u>Manager</u> who <u>Heads</u> a <u>Company</u> must also <u>Works For</u> that <u>Company</u>
13	Each <u>Name</u> which <u>Is Of</u> a <u>Company</u> Must not be <u>Is Of</u> <u>Employee</u>
14	<pre> JAVA.(\$\$RsynonymsOf)= (/* Retrieve the Synonyms of the 'Term' in the 'Context'. Only one level deep, i.e. no Synonyms of Synonyms. Instance Table is not Symmetric but xxxx of it is symmetric.*/ public Vector Retrieve(String Context,String Term) throws Exception { Statement db = DriverManager.createStatement(); ResultSet Synonyms=db.executeQuery(" Select TermLabel1 From Lexon where TermLabel2= '"+ Term + "' and ContextId= '"+ Context + "' and RoleLabel= 'SynonymsOf' union Select TermLabel2 From Lexon where TermLabel1='"+ Term +"' and ContextId= '"+ Context + "' and RoleLabel= 'SynonymsOf'"); ...; return Synonyms; }) </pre>



Example (1) : Interpretation Layer (Rules)

Ontology-base

Lno	Context	Term1	Role	Term2
1	Business	Employee	Has	Name
2	Business	Employee	Works For	Company
3	Business	Manager	Is a	Employee
4	Business	Manager	Heads	Company
5	Business	Company	Has	Name
6	Business	Company	Managed By	Manager
7	Business	Company	Employees	Employee
8	Business	Company	Synonym Of	Business Party
9	Business	Business Party	Synonym Of	Company
10	Business	Name	Is Of	Company
11	Business	Name	Is Of	Employee



Interpretation Layer

RuleID	Rule	ViewID
1	Visible Lexons {1,2,3,4,5,6,7,8,9}	V
2	Each <u>Employee</u> <u>Has</u> at most one <u>Name</u>	V
3	Each <u>Employee</u> <u>Works For</u> at most one <u>Company</u>	V
4	Each <u>Employee</u> <u>Has</u> at least one <u>Name</u>	V
5	Each <u>Company</u> <u>Has</u> at most one <u>Name</u>	V
6	Each <u>Company</u> <u>Has</u> at least one <u>Name</u>	V
7	Each <u>Company</u> <u>Employees</u> at least one <u>Employee</u>	V
8	Each <u>Company</u> <u>Managed By</u> at least one <u>Manager</u>	V
9	Each <u>Company</u> <u>Managed By</u> at least one <u>Manager</u>	V
10	Each <u>Manager</u> who <u>Heads</u> a <u>Company</u> must also <u>Works For</u> that <u>Company</u>	V
11	Visible Lexons {10,11}	V1
12	Each <u>Name</u> which <u>Is Of</u> a <u>Company</u> Must not be <u>Is Of</u> <u>Employee</u>	V1
13	Visible Lexons {8,9}	V2
14	JAVA.(\$RsynonymsOf)= (<i>/* Retrieve the Synonyms of the 'Term' in the 'Context'. Only one level deep, i.e. no Synonyms of Synonyms. Instance Table is not Symmetric but xxxx of it is symmetric.*/</i> public Vector Retrieve(String Context,String Term) throws Exception { Statement db = ConnectionManager.createStatement(); ResultSet Synonyms=db.executeQuery(" Select TermLabel1 From Lexon where TermLabel2= "+ Term + " andContextId= "+ Context + " and RoleLabel= "SynonymsOf" union Select TermLabel2 From Lexon where TermLabel1= "+ Term + " and ContextId= "+ Context + " and RoleLabel= "SynonymsOf;""); ...; return Synonyms; })	V2

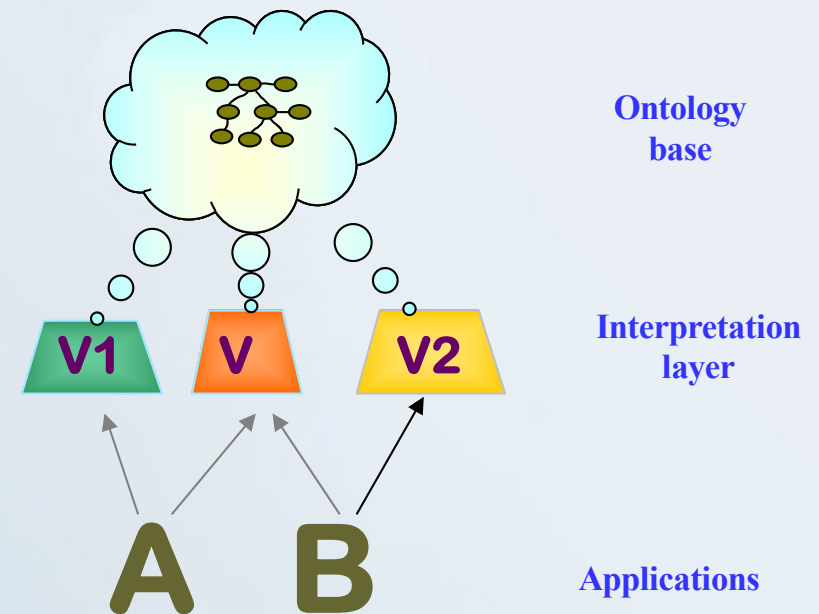


Example (1) : Interpretations (re)use

Interpretation layer $\Leftrightarrow$ **Applications**

Each application can create (reuse/inherit) its interpretational view(s);

Application	Interpretational View
A	V
A	V1
B	V
B	V2



→ Suitable for Semantic Web Environment



Possibilities for Mining o-base and Example (2) : interpretations from different resources

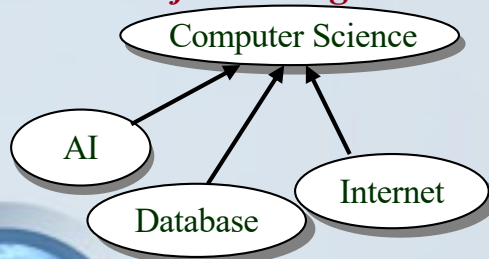
Ontology-base

Context	Term1	Role	Term2
C	Book	Is a	Publication
C	Book	Has	BookID
C	Book	Has	Title
C	Book	Has	Author
C	Book	Has	Price
C	Book	Has	ISBN
C	DataBase	IsSubsubjectOf	Computer Science
C	AI	IsSubsubjectOf	Computer Science
C	Internet	IsSubsubjectOf	Computer Science

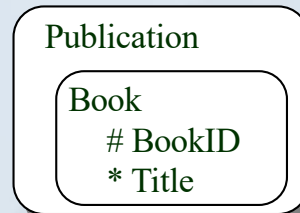
Interpretation Layer (rules)

RuleID	Rule	ViewID
	ER.# (Each <u>Book</u> <u>Has</u> identity by <u>BookID</u>)	
	ER.* (Each <u>Book</u> <u>Has</u> at least one <u>Title</u>)	
	DTD. (Each <u>Book</u> must <u>Has</u> one and only one <u>Title</u>)	
	DTD.+ (Each <u>Book</u> <u>Has</u> at least one <u>Author</u>)	
	DTD.? (Each <u>Book</u> may <u>Has</u> one <u>Price</u>)	
	SQL. Primary Key (Each <u>Book</u> <u>Has</u> <u>ISBN</u> as a primary identity)	
	SQL.NotNull (Each <u>Book</u> <u>Has</u> at least one <u>Title</u>)	

Subjects Categories



ER



DTD

```

<!ELEMENT book(Title, Author+, Price?)>
<!ELEMENT Title (#PCDATA) >
<!ELEMENT Author (#PCDATA) >
<!ELEMENT Price (#PCDATA) >
  
```

SQL

```

Create Table Book(
  ISBN Number Primary Key,
  Title Char(30) NotNull);
  
```

**& Encourage ontologizing
an existing info. systems**



Consistency / Rules systems

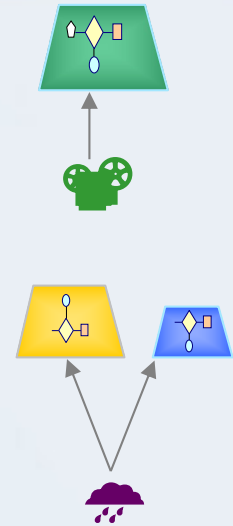
👍 Each interpretational view should be consistent.
(As set of rules that constrain a particular aspect of reality)

👍 Each set of interpretational views used by application should be consistent.

👉 Analyzing the consistency:

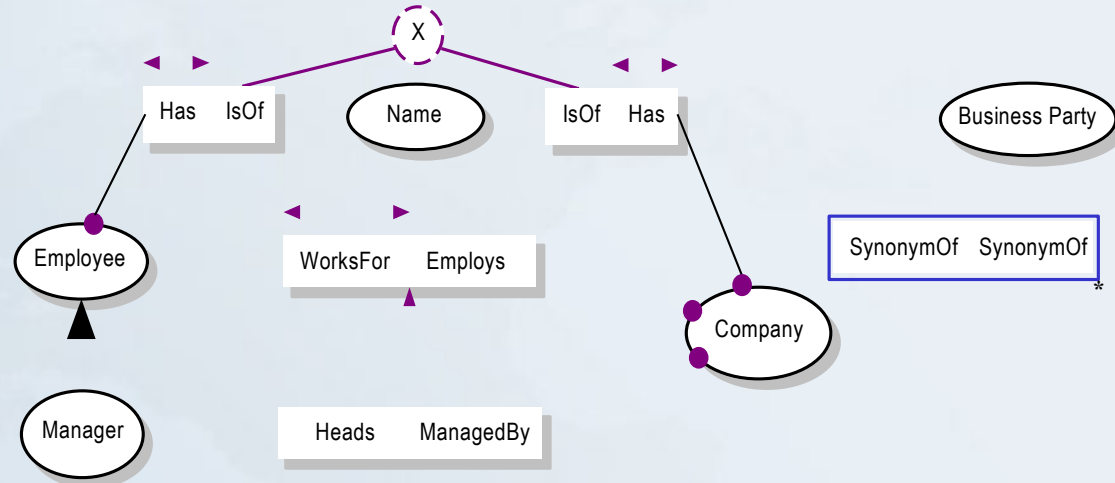
➔ Adopting a given (/well-defined) set of rule types helps analyzing the consistency and evaluating the ontology.

E.g (ORM, Guarino Toolkit, ...)



ORM

Graphical Notation :



ORM Rules :

- A closed set of conceptual constraints for Information systems modeling.

[Verheyen, & Bekkum, 82]

- Frame logic-like, supporting “Is a” relationship.

- **Mapable** to first order logic.

- Consistency Analyzer, see RIDL-A [Meersman, 88]

- Tools, see [VisioModeler, [Microsoft](#)], [DogmaModeler, [STARLab](#)]

- More, see [Halpin, 2001], [<http://www.orm.net>]



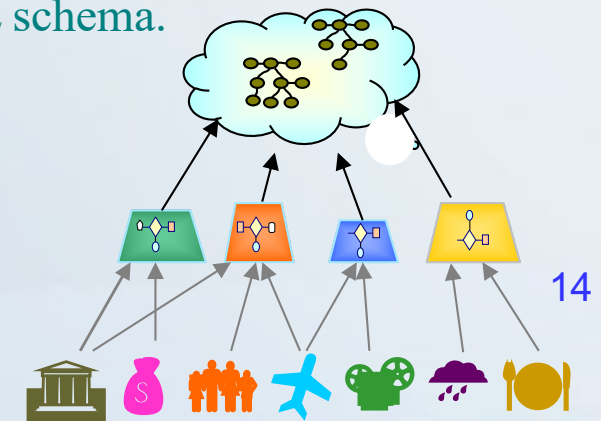
Rules Systems / interpretation server

Allowing several rules systems (languages/standards)

- ✓ Speeds up the interpretation building and user scenario specifications
- ✓ Encourage ontologizing an existing information system, [Meersman,2001]
- ✓ Using Natural Language templates / formats as a fixed syntax to write for rules modeling; allow non-experts to (help) check, validate or build the interpretation rules.

➔ This implies an *interpretation server* / *Rule Engine* to store, serve and map between them:

- See [Halpin, 2001] for mapping between ORM, EER, XML Schema, and the UML.
- See [BCFK, 99] for mapping between UML and XML schema.



Conclusion

- ✎ Increase independence between ontology and application.
 - ✓ Intended Conceptualizations & Interpretation layer
- ✎ Accelerating and simplifying ontology construction.
 - ✓ Speeding up the ontology building/mining processes. Agreements are much easier
- ✎ Building ontology-based business rules.
- ✎ Rules languages and standards should be NL-like, and support legacy standards (e.g. SQL, ER, Java, ...).
 - ✓ Allowing heterogeneous rule systems (help: mining, ontologizing, analyzing,...)
 - ✓ By non-programmer business domain experts. (e.g. NL templates)
- ✎ Managing communication between ontology and applications.
 - ✓ Each application can create (reuse/inherit) its interpretational view(s);
- ✎ Management of large scale ontology bases.
 - ✓ Easily store ontology-base (lexons) & interpretation layer (rules) in DB
- ✎ Versioning, scalability ,...
 - ✓



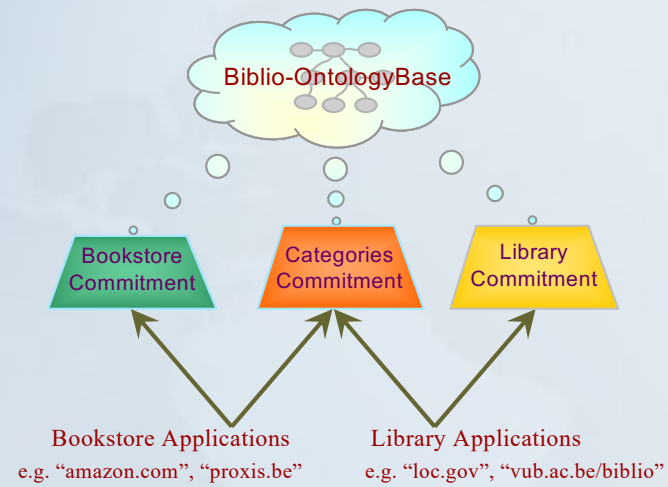
References

- [BCFK,99] G. Booch, M. Christerson, M. Fuuchs, J. Koistinen. UML for XML schema mapping specification. Technical report, Rational Software, 1999.
- [Halpin, 2001] Halpin, T. "Information Modeling and Relational Databases", 3rd Ed., Morgan-Kaufmann, 2001.
- [Guarino,98] Guarino, N. 1998. Some Ontological Principles for Designing Upper Level Lexical Resources. *Proceedings of LREC-98*.
- [Meersman,2001] Meersman, R., "Ontologies and Databases: More than a Fleeting Resemblance", in: "Proceedings of the 2001 OES-SEO Workshop, Rome", Missikoff, M. (ed), LUISS Publications, 2001.
- [Meersman, 88] Meersman R., De Troyer O., Verlinden P., "RIDL* on the cris case: a workbench for NIAM", in: "Proceedings of the IFIP WG 8.1 Working Conference on Computerized Assistance during the information systems life cycle, 1988.
- [Verheyen&v.Bekkum,,84] Verheyen, G. and van Bekkum, P.: NIAM, aN Information Analysis Method. In: IFIP Conference on Comparative Review of Information Systems Methodologies, T.W. Olle, H. Sol, and A. Verrijn-Stuart (eds.), North-Holland (1982).
- [DogmaModeler] <http://www.starlab.vub.ac.be/dogma>
- [VisioModeler] [http://www.microsoft.com/downloads/...\(downloadable\)](http://www.microsoft.com/downloads/...(downloadable))
- [ORM] <http://www.orm.net>



Thank You





**Interpretation
layer**

Applications



